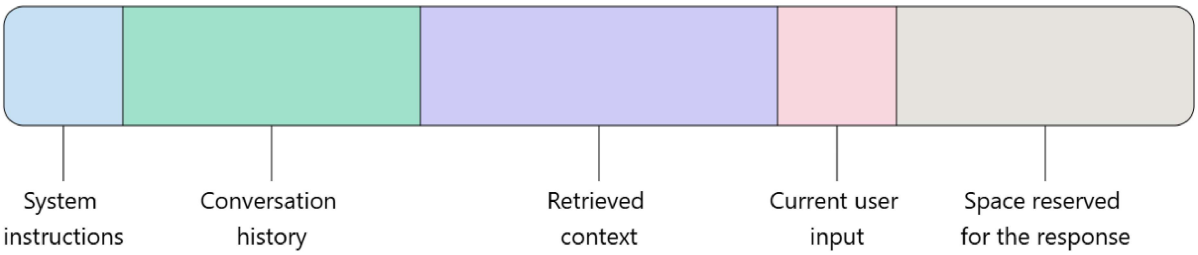
What Goes Into the Context Window: A Reference

Introduction

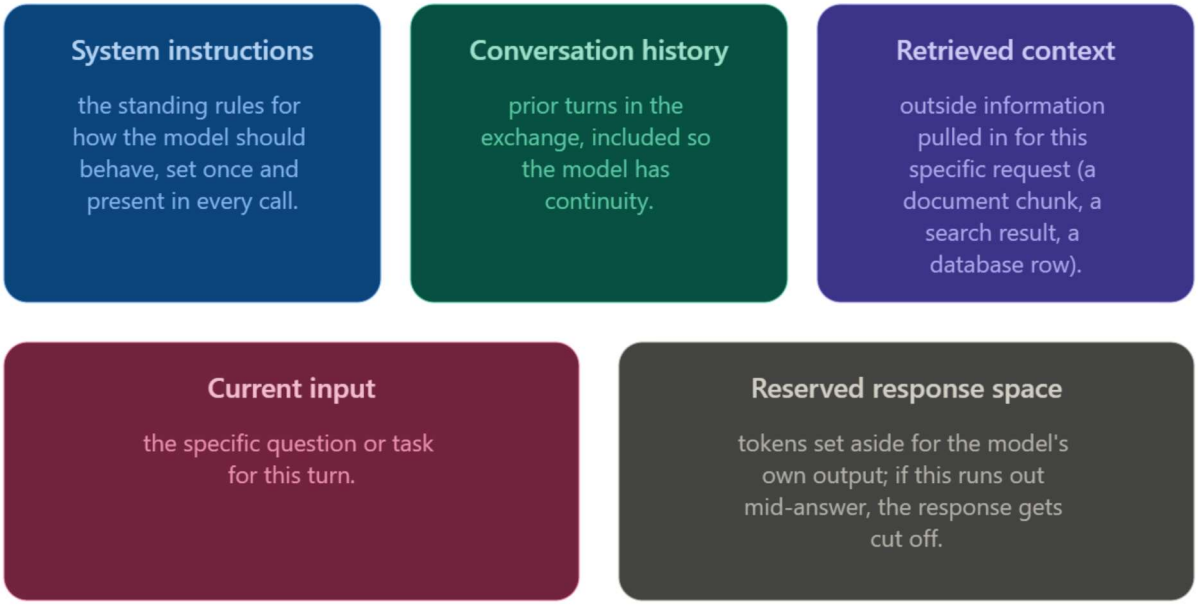
Everything a model can "see" when generating a response lives inside a single boundary: the context window. This reading is a reference for what actually fills that space — the pieces that compete for room, and why managing them well changes how reliable an application's output is.

What the context window actually is

The context window is the total amount of text (measured in tokens) a model can process in a single call — everything you send in, plus everything it generates back, has to fit inside this one shared budget.



The pieces that fill it



Why order and priority matter

Not all of these compete equally for space. A budget that's too tight forces trade-offs, and some trade-offs hurt more than others.

If this gets trimmed...	The effect is...
System instructions	Behavior drifts from the intended rules
Conversation history	The model "forgets" earlier parts of the exchange
Retrieved context	Answers lose grounding in the specific source material
Current input	Rare, but truncating this loses the actual question
Reserved response space	Output gets cut off mid-generation

As a rule, system instructions and current input are the last things to sacrifice — history and retrieved context are usually where trimming happens first, since they're the most compressible without breaking the task entirely.

A worked example

```
System: "You are a support assistant. Be concise."          ~15 tokens
History: 6 prior turns of conversation                      ~400 tokens
Retrieved: 2 relevant help-doc chunks                       ~600 tokens
Current input: "How do I reset my password?"                ~10 tokens
Reserved for response                                       ~200 tokens
```

Total budget: 4,000 tokens → plenty of room here.

> If retrieved context grew to 3,500 tokens instead, something else would need to shrink to keep the total in bounds.

Summary

A shared, finite budget

the context window is a shared, finite budget — not unlimited memory.

Five parts compete

instructions, history, retrieved context, input, and reserved output space.

Trim wisely

trim the most compressible pieces first — usually history and retrieved context, not instructions or the current question.

Silent drift is worse

running out mid-response is the most visible failure mode, but silent behavior drift from trimmed instructions is often worse.